hashlock.

Security Audit

UNIT Technologies (RWA)

June 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The UNIT Technologies team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

UNIT is a decentralized finance (DeFi) protocol designed to make stablecoin liquidity productive through a tokenized deposit and staking model. The protocol allows users to deposit supported stablecoins and receive **UNIT**, a stable asset backed 1:1 by the underlying stablecoins deposited into the protocol. UNIT serves as the primary liquid asset of the ecosystem and can be held, transferred, or used within supported DeFi applications.

The protocol separates the liquid representation of deposited assets from participation in yield generation through **sUNIT (Staked UNIT)**. Users can stake UNIT to receive sUNIT, which represents their staked position in the protocol and provides exposure to rewards generated from the underlying stablecoin capital. This structure allows UNIT to remain liquid while users who choose to stake can participate in the yield generated by the protocol.

Project Name: UNIT Technologies

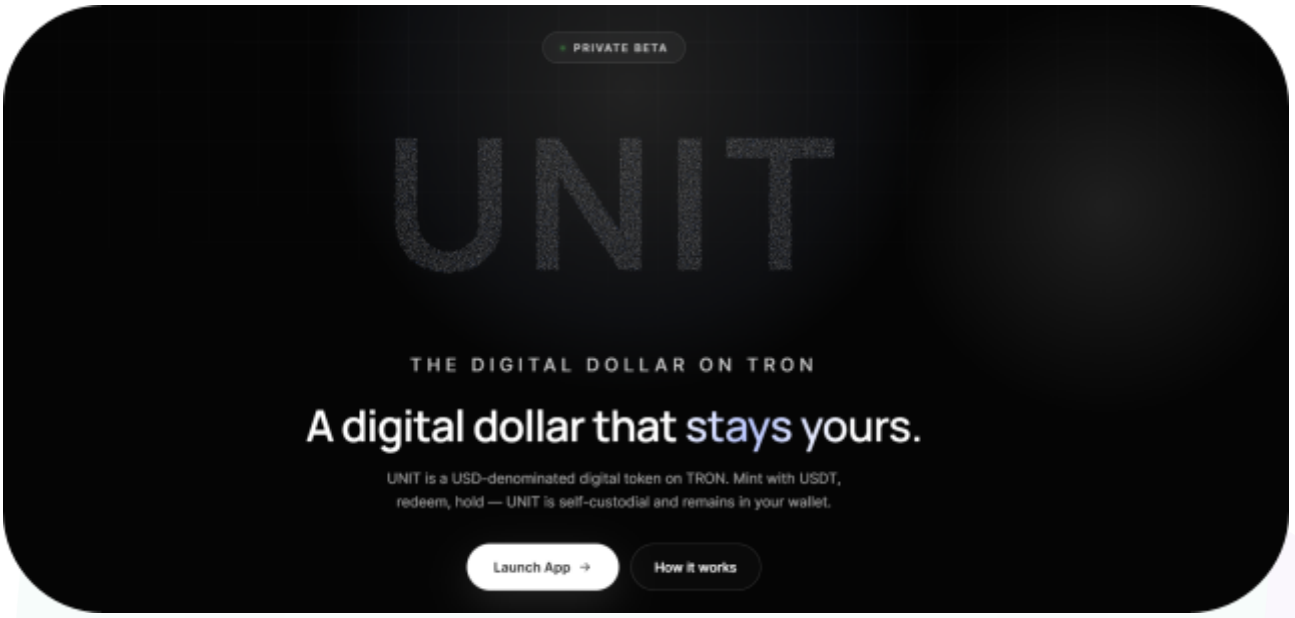
Project Type: RWA

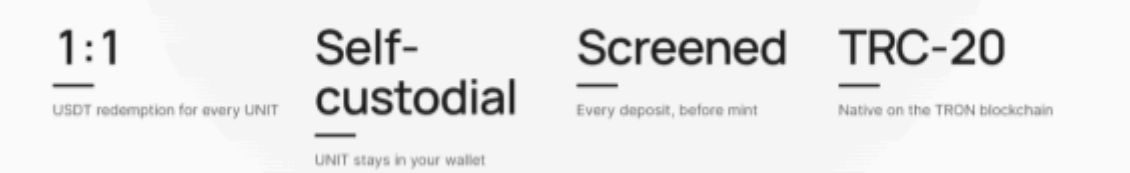
Compiler Version: ^0.8.26

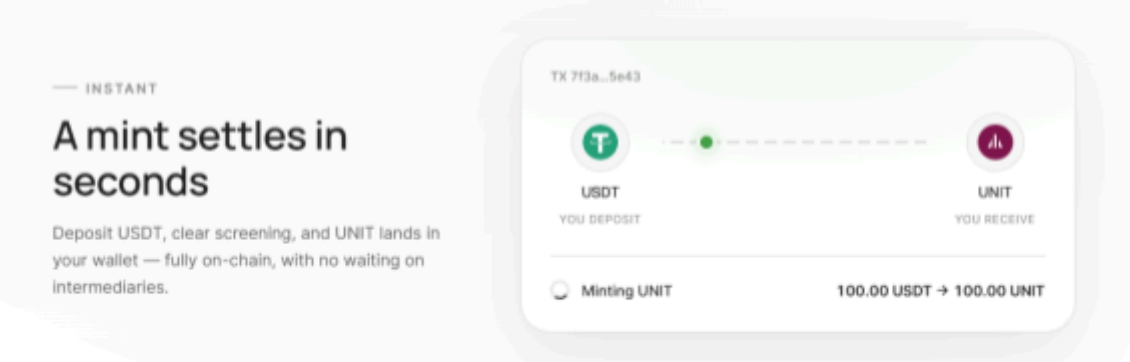
Website: unitdefi.com

Logo:

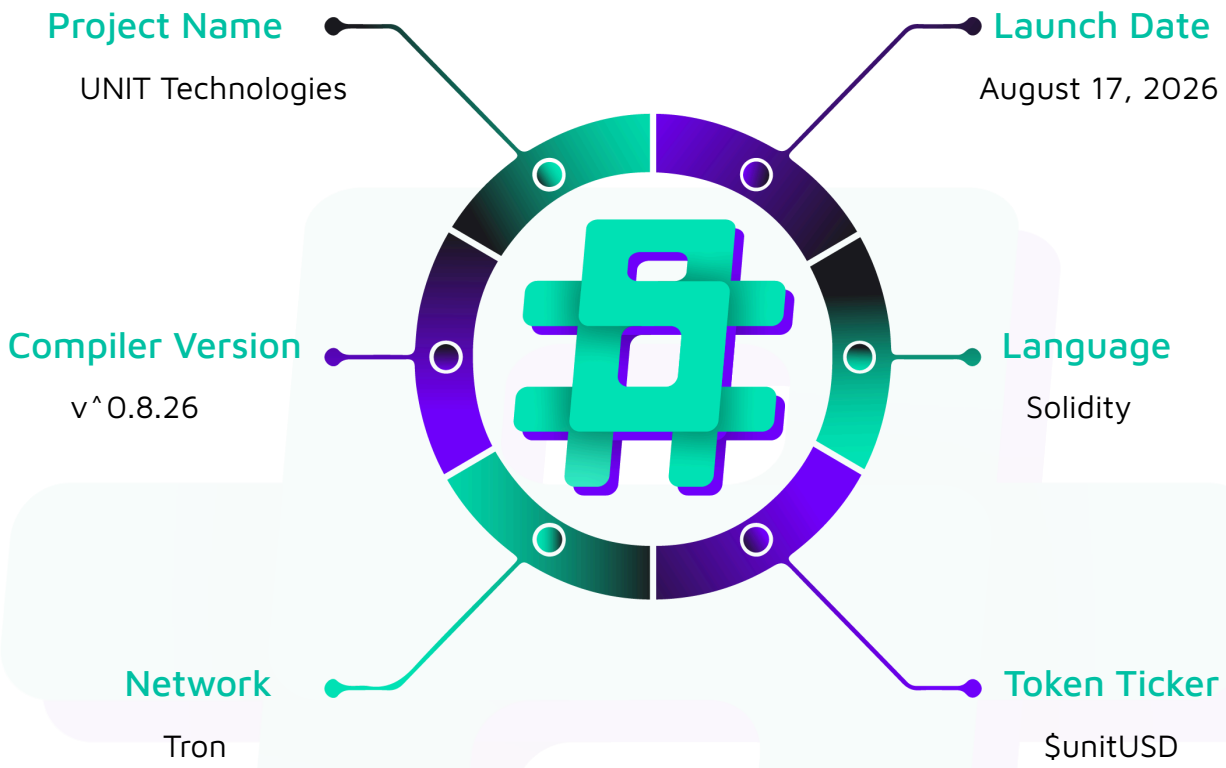
Project Visuals:







Visualised Context:



Audit Scope

We at Hashlock audited the solidity code within the UNIT Technologies project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**UNIT Technologies Smart Contracts**
Network	**TRON**
Language	**Solidity**
Audit Date	**June, 2026**
Contract 1	Minter.sol
Contract 2	StakedUnit.sol
Contract 3	Unit.sol
Audited GitHub Commit Hash	2060b65caa74662beaa5ad8cba9663251a2536d4

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

5 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the smart contracts of the UNIT Technologies project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the UNIT Technologies project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Minter - Malicious Signer Can Drain All Pending Redeems (Redeem Signature Not Tied to Caller)

Description

Unlike `mint()`, the `redeem` function takes an arbitrary `account` parameter and does not bind the caller (`msg.sender`) to the signed payload. A compromised or malicious `SIGNER_ROLE` holder can sign a redeem for any user who has `pendingRedeems > 0` and direct the USDT to any address (including the attacker's).

Vulnerability Details

```
// Minter.sol
function redeem(address account, uint256 assets, address signer, uint256 deadline,
bytes calldata signature)
    external
{
    _checkPermit(
        signer,
        _hashTypedDataV4(keccak256(abi.encode(REDEEM_TYPEHASH, account, assets,
_useNonce(account), deadline))),
        deadline,
        signature
    );
    if (pendingRedeems[account] < assets) revert InsufficientPendingRedeem();
    pendingRedeems[account] -= assets;
    if (USDT.balanceOf(address(this)) < assets) revert InsufficientBalance();
    USDT.safeTransferFrom(address(this), account, assets);  // note: transfers to
`account`, but attacker controls signature
    emit Redeemed(account, assets);
}
```

The `REDEEM_TYPEHASH` does not include `msg.sender` or a `recipient` field. Any holder of a valid signature (leaked via mempool, compromised frontend, or malicious signer) can front-run and steal the redemption.

Impact

Direct theft of user redemptions. A compromised signer can drain the entire USDT balance held by the Minter contract by signing large redeems for all users with pending amounts. Medium severity because it requires signer compromise or signature leakage, but impact is direct fund loss.

Recommendation

- Add `recipient` parameter to the redeem typed struct and transfer USDT to it instead of always to `account`.
- Bind the caller: change to `redeem(uint256 assets, address signer, uint256 deadline, bytes calldata signature)` and use `msg.sender` as the account (require user to call directly).
- Or add a two-step claim where only the account owner can pull after a signed approval.
- Consider adding `recipient` to the typehash and update tests/off-chain signing logic.

Note

Hashlock agrees wit UNIT claims at one point. In `redeem` the USDT is sent to `account`, the address bound in the signed payload, so a caller cannot redirect it to an address of their choosing, and this is not third party theft. Hashlock believes the finding remains valid for a different reason. `redeem` binds `account` in the signature but not the caller and performs no caller authentication, so a compromised or malicious `SIGNER_ROLE` holder can force the withdrawal of a legitimate user's redeemable USDT without that user's authorization. Since the user's UNIT is already burned at that point, the user cannot prevent or control the movement of the USDT owed to them. Hashlock rates this Medium because it requires signer compromise, and it is not invalid. The recommended fix is to bind the caller in `redeem`.

Status

Acknowledged

Low

[L-01] StakedUnit, Unit - Confiscate Breaks Vault Accounting (Bidirectional)

Description

`confiscate()` on the vault address or direct transfers cause `totalAssetBalance` to desync from real balance.

Vulnerability Details

```
// StakedUnit.sol: totalAssets() reads the internal accumulator, never tracks the
actual token balance inside the contract
function totalAssets() public view override returns (uint256 assets) {
    assets = totalAssetBalance;
    ...
}

// Unit.sol: confiscate has no carve-out for the vault address
function confiscate(address from, address to, uint256 value) external
onlyRole(CONFISCATOR_ROLE) {
    _transfer(from, to, value);   // can move the vault's unitUSD in or out
}
```

`totalAssets()` and `_transferIn/_transferOut` only update the internal accumulator `totalAssetBalance`; they never reconcile against the vault's real `Unit.balanceOf(address(this))`. So any movement of the vault's `unitUSD` through a path other than deposit/withdraw is invisible to share pricing.

Impact

Insolvency for remaining users or permanently stranded surplus tokens. Severity is Low because every path that creates the divergence requires the privileged `CONFISCATOR_ROLE` (or a voluntary donor) rather than an unprivileged attacker; the consequence of the resulting withdrawal race, however, is unprivileged.

Recommendation

Add a `reconcile()` or `recover()` function, and exclude the vault address from `confiscate`, only the amount outside the `totalAssets()`

Status

Acknowledged

[L-02] StakedUnit, Minter, Unit - `DEFAULT_ADMIN_ROLE` Renounce Bricks All Contracts

Description

Admin can renounce `DEFAULT_ADMIN_ROLE`, permanently disabling role management, rate setting, and confiscation. No override of `renounceRole` for `DEFAULT_ADMIN_ROLE`.

Impact

Permanent loss of control. No way to restore MINTER_ROLE, CONFISCATOR_ROLE, or call `setRate()`. Funds can become frozen.

Recommendation

Add `renounceRole` override that reverts when `role == DEFAULT_ADMIN_ROLE`. Implement 2-step admin transfer pattern.

Status

Acknowledged

[L-03] Minter - Fee-on-Transfer (FOT) Token Breaks Minter Invariant

Description

Minter assumes `assets` transferred = `assets` minted. FOT tokens break this.

Vulnerability Details

```
// Minter.sol
function mint(uint256 assets, address custody, address signer, uint256 deadline,
bytes calldata signature)
    external
{
    ...
>>> USDT.safeTransferFrom(msg.sender, custody, assets);
    UNIT.mint(msg.sender, assets);
    emit Minted(msg.sender, custody, assets);
}
```

Impact

High on TRON deployment. TRON USDT has historically had fee-on-transfer capability (even if currently disabled). If enabled, or if any future USDT upgrade introduces fees, the Minter will mint more `unitUSD` than USDT received, leading to systemic under-collateralization.

Recommendation

- Immediately measure balance delta (`balanceOf` before/after `safeTransferFrom`) and mint only the actual received amount.
- Add an explicit check or admin-gated flag to disable minting if the underlying asset is known to be fee-on-transfer.
- Strongly consider adding a formal invariant test that `unitUSD` supply never exceeds total USDT held across custody + Minter.

Status

Acknowledged

[L-04] StakedUnit - lastUpdate Is Updated Even When No Yield Is Minted

Description

The `_sync()` function updates `lastUpdate` before calculating and minting yield. This allows an attacker to artificially advance the timestamp and cause yield truncation on subsequent calls.

Vulnerability Details

```
// StakedUnit.sol
    function _sync() private {
        uint256 timeElapsed = block.timestamp - lastUpdate;
        if (timeElapsed > 0) {
>>>         lastUpdate = block.timestamp;
            uint256 yield = (totalAssetBalance * rate * timeElapsed) / (BPS * 365
days);
            if (yield > 0) {
                totalAssetBalance += yield;
                Unit(address(asset())).mint(address(this), yield);
            }
        }
    }
```

Impact

Frequent calls to `deposit(0)` or `setRate(0)` can permanently suppress yield accrual for the vault. The client README claims they will maintain a minimum TVL of 100,000 unitUSD to make this griefing "not possible". However, there is no on-chain enforcement of this minimum TVL. Therefore the griefing vector remains viable if TVL ever drops (or during early deployment).

Recommendation

- Only update `lastUpdate` if `yield > 0`.
- Implement a small remainder accumulator to carry forward fractional yield.
- Consider adding an on-chain `minimumTVL` guard that pauses yield if balance falls below threshold (or document this as an off-chain operational requirement).

Status

Acknowledged

[L-05] StakedUnit - Multiple ERC-4626- Specification Violations in totalAssets() and Preview Functions

Description

The StakedUnit vault violates several core requirements of the ERC-4626 standard due to its virtual yield mechanism in totalAssets(), high decimal offset, and manual accounting. While not immediately exploitable for fund loss in all scenarios, these violations break integrator expectations and create edge-case failures.

Vulnerability Details

```
// StakedUnit.sol
function totalAssets() public view override returns (uint256 assets) {
    assets = totalAssetBalance;
    uint256 timeElapsed = block.timestamp - lastUpdate;
    if (timeElapsed > 0 && assets > 0) {
        assets += (assets * rate * timeElapsed) / (BPS * 365 days);
    }
}

// StakedUnit.sol
function _decimalsOffset() internal view virtual override returns (uint8) {
    return 12;
}
```

This breaks:
- previewWithdraw which MUST return no fewer shares than actual execution.
- convertTo functions which must not reflect on-chain timing/slippage.
- maxWithdraw that can overestimate withdrawable assets.

Impact

Integrators cannot reliably use preview functions for accurate slippage or withdrawal calculations. The last remaining user cannot withdraw their full entitled balance, leaving permanent dust locked in the contract. This breaks multiple MUST requirements in the ERC-4626 specification regarding preview consistency, rounding behavior, and

maximum withdrawable amounts. A compromised or griefing actor can indirectly exploit timing to cause failed withdrawals or inaccurate pricing.

Recommendation

- Replace the virtual yield mechanism in `totalAssets()` with a proper accrual index (similar to Compound or Aave).
- Implement a custom `maxWithdraw` / `previewWithdraw` override that handles the final redemption edge case.
- Add comprehensive tests for sole-holder redemption scenarios and virtual yield timing attacks.

Status

Acknowledged

QA

[Q-01] Minter - Inconsistent and Unnecessary Use of safeTransferFrom + Self-Approval

Description

Two related minor issues exist in `Minter.sol`:

1. In `redeem()` the code uses `safeTransferFrom(address(this), account, assets)` even though `from == msg.sender`. This should use `safeTransfer(account, assets)` instead (slightly more gas efficient and clearer).

2. The constructor does `USDT.forceApprove(address(this), type(uint256).max)`. This self-approval is completely unnecessary because `redeem()` does not rely on any allowance when transferring from the contract itself (`from == msg.sender` bypasses the allowance check entirely, even on tokens with approval race protection).

Vulnerability Details

```
// Constructor
USDT.forceApprove(address(this), type(uint256).max);

// redeem()
USDT.safeTransferFrom(address(this), account, assets); // should be safeTransfer
```

Impact

Low / QA. Minor gas waste, reduced code clarity, and confusing state left on-chain for monitoring tools. No direct security impact, but represents poor coding practices.

Recommendation

- Remove the `forceApprove` line from the constructor entirely.
- Change the line in `redeem()` to `USDT.safeTransfer(account, assets);`.
- Update the inline comment.

Both changes are trivial, improve cleanliness, and should be accompanied by a test to confirm `redeem()` still works correctly.

Status

Acknowledged

Centralisation

The UNIT Technologies project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the UNIT Technologies project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au